

עבודה להגשה 1

משתתפים: עדן קרייניץ - 211589742
איתי רוזן - 207931437

חלק א' - פיתוח פיזיקלי ומתמטי:

1. פירוק כוחות:

כוח הכבידה הפועל כלפי מטה בציר ה-y: $-mg\hat{y}$
כוח התנגדות האוויר הפועל בכיוון מנוגד לכיוון התנועה: $-(bv + cv^2)\hat{v}$

נזכיר כי $\hat{v} = \frac{\vec{v}}{v}$, ולאחר הצבה נקבל: $-(b + cv)\vec{v}$
רכיב x ו-y של הכוח: $f_x = -(b + c \cdot \sqrt{v_x^2 + v_y^2})v_x$
 $f_y = -(b + c \cdot \sqrt{v_x^2 + v_y^2})v_y$

2. תאוצה:

$$a_x = -\frac{1}{m} \left(b + c \cdot \sqrt{v_x^2 + v_y^2} \right) v_x$$

$$a_y = -\frac{1}{m} \left(b + c \cdot \sqrt{v_x^2 + v_y^2} \right) v_y$$

חלק ב' - כתיבת קוד לפי השלב הקודם:

```
import numpy as np
import matplotlib.pyplot as plt

# variables
m = 0.1
g = 9.8
b = 0.2
c = 0.3
h = 0.5
v0 = 10
alpha = 30 * (np.pi / 180)
dt = 0.001

# starting conditions
x = 0
y = h
vx = v0 * np.cos(alpha)
vy = v0 * np.sin(alpha)
t = 0

# empty lists to then put data from each iteration in
xList = [x]
yList = [y]
vyList = [vy]
tList = [t]

# "euler calculation".. runs until the ball hits the floor
while y >= 0:
    # current speed
    v = np.sqrt(vx**2 + vy**2)

    # acceleration
    ax = -(1 / m) * (b + c * v) * vx
    ay = -g - (1 / m) * (b + c * v) * vy

    # update next speed
    vx = vx + ax * dt
    vy = vy + ay * dt

    # update next position
    x = x + vx * dt
```

```

    y = y + vy * dt

    # update time
    t = t + dt

    # update the lists
    xList.append(x)
    yList.append(y)
    vyList.append(vy)
    tList.append(t)

# update the "end stats"
maxHeightTime = tList[yList.index(max(yList))]

returnHeightTime = 0
for i in range(len(yList)):
    if yList[i] <= h and yList[i] <= h:
        returnHeightTime = tList[i]
        break

totalTime = tList[-1]
horizontalRange = xList[-1]

# parabola fit
# calculates the A, B, and C coefficients for the bestfit parabola
coefficients = np.polyfit(xList, yList, 2)
# plugs the x values into the new parabola equation to generate a list of y
values without air resistance
yPara = np.polyval(coefficients, xList)

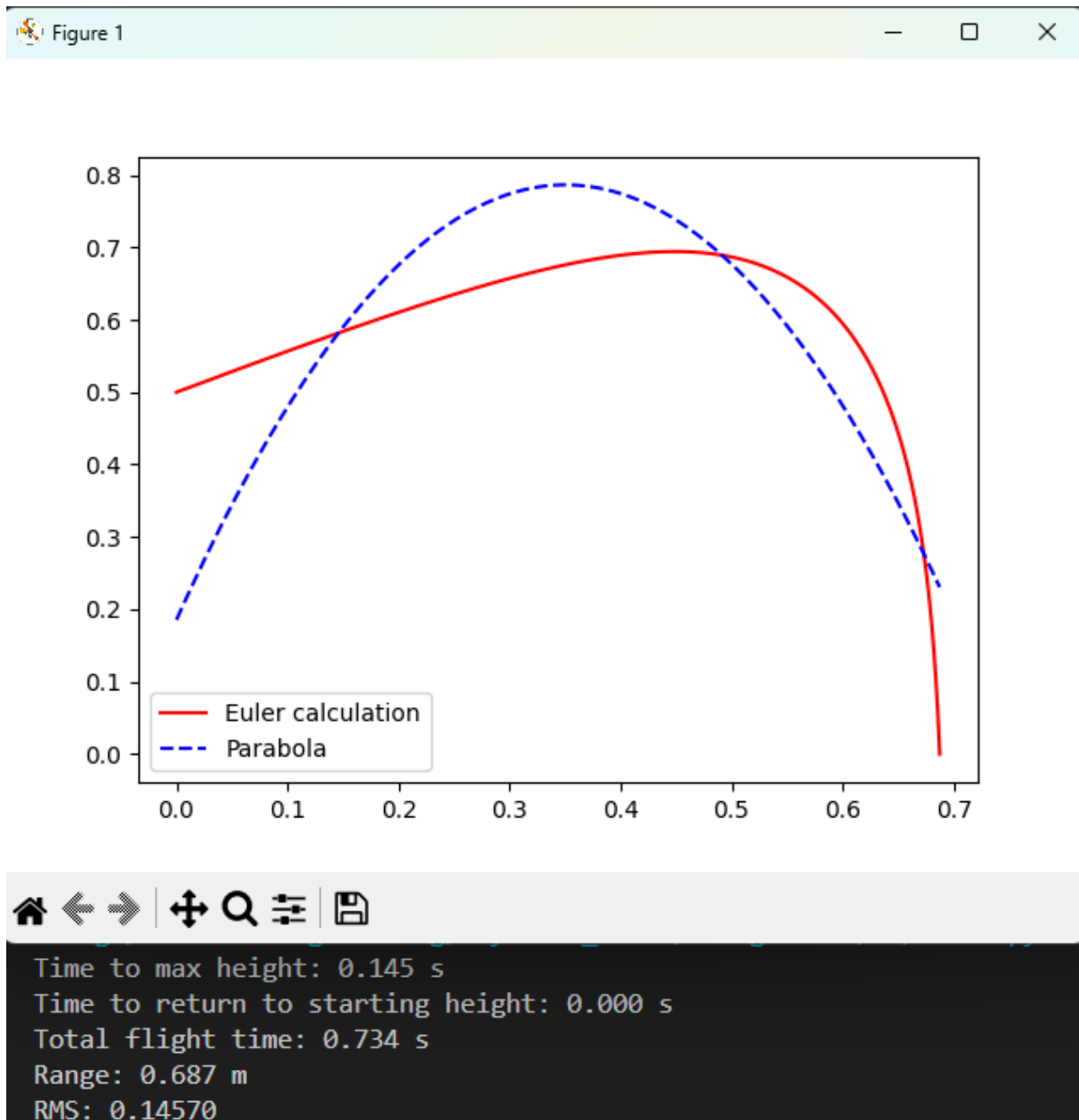
# RMS calculation
# converts the list of simulated y values into a NumPy array, and parabola y
values
yArr = np.array(yList)
yParaArr = np.array(yPara)
# calculates the RMS
rms = np.sqrt(np.mean((yArr - yParaArr) ** 2)) / (np.max(yArr) - np.min(yArr))

# display the stats
print(f"Time to max height: {maxHeightTime:.3f} s")
print(f"Time to return to starting height: {returnHeightTime:.3f} s")
print(f"Total flight time: {totalTime:.3f} s")
print(f"Range: {horizontalRange:.3f} m")

```

```
print(f"RMS: {rms:.5f}")

# graph
plt.plot(xList, yList, label="Euler calculation", color="red")
plt.plot(xList, yPara, label="Parabola", color="blue", linestyle="--")
plt.legend()
plt.show()
```



חלק ג' - הקוד של ג'מיני:

פרומפט:

צירפתי לשיחה שלי עם ג'מיני את קובץ העבודה....:

I've attached a PDF file to this message.

here's the code I've written for this assignment:

“my code”

I want you to go down to "חלק ג'" and give me a "NiceGUI app" as per the guidelines for this assignment.

Make sure you stick as close as possible to the assignment "asks" and "wants".

Also tell me if I'm missing something

ג'מיני ציין כי יש לי כמה טעויות לוגיות בקוד,
כמו: חישוב זמן החזרה לגובה c , יישום שגוי של RMS , ועוד כמה טעויות סינטקטיות...
אצרף לעבודה זו את הקוד שג'מיני כתב.

חלק ד' - תיאור העבודה:

תחילה פרסתי את כל הנתונים שלי על הטאבלט על מנת לחשב את כל הנוסחאות שאליהן אזדקק בזמן העבודה.

ברגע שהאמנתי כי היו לי את כל הנתונים הנחוצים התחלתי לכתוב את הקוד שלי בעזרת תיווך של בינה מלכותית (לא ביקשתי פתרונות, אלא הכוונה... באופן דומה לעבודה הראשונה, לא הצלחתי להבין מה עלי לעשות, לכן ביקשתי מבינה מלכותית לפרק לי את העבודה לשלבים).

NiceGUI.משם המשכתי לעבודה מול ג'מיני עבור

נאלצתי לשלוח מספר עוד 3 הודעות לבינה על מנת שסוף סוף היא הצליחה להשיג את התוצאה הרצויה.